

Review of Research Paper number 24

author 1, author 2, author 3

version: March 17, 2026

1 Introduction

1.1 Summary of the paper

The article focuses on privacy preserving *via* encryption (which is one of three ways among anonymisation, encryption, and differential privacy). It mainly says that existing approaches of encryption are cloud-based solution. Thus, the authors have the original idea to combine peer-to-peer (what they call cooperative) ADMM with encryption.

1.2 Problem formulation

We consider typical setting, introduced in the course, for ADMM application :

$$\begin{aligned} \min_{x^i \in \mathbb{R}^n, y^{ij} \in \mathbb{R}^{|\mathcal{E}|}} \quad & \sum_{i=1}^N f_i(x^i) \\ \text{s.t } \quad & x^i = y^{ij}, x^j = y^{ij} \quad \forall i \sim j \\ & G_i x^i = E_i p_i \quad \forall i \end{aligned} \tag{1}$$

where

$$f_i(x^i) = \frac{1}{2}(x^i)^T H_i x^i + p_i^T F_i^T x^i \tag{2}$$

The authors also supposes that one has the particular following structure: $x^i = \begin{pmatrix} \alpha_i \\ \gamma_i \end{pmatrix}$, and $p_i = \begin{pmatrix} \beta_i \\ \delta_i \end{pmatrix}$.

For each agent α_i is the internal data and γ_i represents information it has over its neighbors. δ_i is set by the operator and β_i by the agent.

The problem formulation consists in finding a peer-to-peer algorithm which fulfilled the following assumptions:

1. Only agent i should have access to $\alpha_i, \beta_i, H_i, F_i, G_i$ and E_i .
2. Only the operator should have access to δ_i .
3. Neither devices, nor the operator should have access to γ_i .
4. Neither devices, nor operator should have access to y^{ij} .

1.3 Assumptions

Appart the definition of the function for each agent, there is no particular assumptions. We are in the good setting of convex, strongly convex (if H_i is positive definite) and smooth function, so linear convergence holds for peer-to-peer ADMM.

2 Result: theory and practice

2.1 Theory

Let derive the augmented Lagrangian for each device :

$$\mathcal{L}_i(x^i, y, \lambda) = f_i(x^i) + (\tilde{\lambda}^i)^T (G_i x^i - E_i p_i) + \frac{\beta}{2} \|G_i x^i - E_i p_i\|^2 + \sum_{j \sim i} (\lambda^{ij})^T (x^i - y^{ij}) + \frac{\beta}{2} \|x^i - y^{ij}\|^2 \quad (3)$$

One can compute the gradient in order to derive ADMM iterations:

$$\nabla_{x^i} \mathcal{L}_i(x^i, y, \lambda) = H_i x^i + F_i p_i + (\tilde{\lambda}^i)^T G_i^T + \beta G_i^T (G_i x^i - E_i p_i) + \sum_{j \sim i} [\lambda^{ij} + \beta(x^i - y^{ij})] \quad (4)$$

$$= (H_i + \beta(\deg(x^i)I + G_i^T G_i))x^i + F_i p_i + G_i^T \tilde{\lambda}^i - \beta G_i^T E_i p_i + \sum_{j \sim i} [\lambda^{ij} - \beta y^{ij}] \quad (5)$$

So one can derive the ADMM iterations:

- **Computation step**

$$x_{k+1}^i = (H_i + \beta(\deg(x_k^i)I + G_i^T G_i))^{-1} \left(\beta G_i^T E_i p_i - F_i p_i + -G_i^T \tilde{\lambda}^i - \sum_{j \sim i} [\lambda^{ij} - \beta y^{ij}] \right) \quad (6)$$

- **Communication step** Each device communicates x_{k+1}^i to its neighbors.

- **Computation step**

$$y_{k+1}^{ij} = \frac{x_{k+1}^i + x_{k+1}^j}{2} \quad (7)$$

$$\lambda_{k+1}^{ij} = \lambda_k^{ij} + \beta(x_{k+1}^i - y_{k+1}^{ij}) \quad (8)$$

$$\tilde{\lambda}_{k+1}^i = \tilde{\lambda}_k^i + \beta(G_i x_{k+1}^i - E_i p_i) \quad (9)$$

The main result of the algorithm is to introduce the following encrypted version of ADMM:

- **Computation step**

$$\llbracket x_{k+1}^i \rrbracket_0 = (H_i + \beta(\deg(x_k^i)I + G_i^T G_i))^{-1} \odot \left(\beta G_i^T E_i \odot \llbracket p_i \rrbracket_0 \ominus F_i \odot \llbracket p_i \rrbracket_0 \ominus G_i \odot \llbracket \tilde{\lambda}^i \rrbracket_0 \ominus \bigoplus_{j \sim i} [\llbracket \lambda^{ij} \rrbracket_0 \ominus \llbracket \beta y^{ij} \rrbracket_0] \right) \quad (10)$$

- **Communication step** Each device communicates $\llbracket x_{k+1}^i \rrbracket_0$ to its neighbors.

- **Computation step**

$$\llbracket y_{k+1}^{ij} \rrbracket_0 = \frac{1}{2} \odot (\llbracket x_{k+1}^i \rrbracket_0 \oplus \llbracket x_{k+1}^j \rrbracket_0) \quad (11)$$

$$\llbracket \lambda_{k+1}^{ij} \rrbracket_0 = \llbracket \lambda_k^{ij} \rrbracket_0 + \beta \odot (\llbracket x_{k+1}^i \rrbracket_0 \ominus \llbracket y_{k+1}^{ij} \rrbracket_0) \quad (12)$$

$$\llbracket \tilde{\lambda}_{k+1}^i \rrbracket_0 = \llbracket \tilde{\lambda}_k^i \rrbracket_0 + \beta \odot (G_i \odot \llbracket x_{k+1}^i \rrbracket_0 \ominus \llbracket E_i p_i \rrbracket_0) \quad (13)$$

2.2 Practice

Code the main algorithm of the paper and apply it to the kernel setting of your python project. Compare the result with the most similar algorithm from the course. Comment on the results. [1 page max] [Send a link/file to the python code].

3 Conclusion

Draw some conclusion to put the paper in perspective with the course. [10 lines]